

LABORATÓRIO PTHREADS

Faculdade de Computação e Informática - FCI

Aluno: Pedro de Souza Zequi

Turma: 05N11

RA: 10419805

Repositório: <https://github.com/PedroSZequi/CompParalela>

1. Introdução e Fundamentos Teóricos

Este laboratório teve como objetivo observar, na prática, conceitos de programação concorrente em memória compartilhada utilizando POSIX Threads (Pthreads). Foram estudados criação e sincronização de threads, condições de corrida, exclusão mútua com mutex, divisão de trabalho e análise de desempenho por meio de Speedup e Eficiência.

Também foi analisado o comportamento de threads em Python (CPython) em uma tarefa CPU-bound. Nesse caso, o foco foi entender o efeito do Global Interpreter Lock (GIL), que limita a execução simultânea de bytecode Python por múltiplas threads quando está habilitado.

2. Metodologia e Ambiente Experimental

Os códigos foram desenvolvidos e executados no GitHub Codespaces. Os programas em C foram compilados com as flags -O2 -Wall -pthread. Para os testes de desempenho foram realizadas cinco execuções por configuração, usando 1, 2, 4 e 8 threads quando aplicável. O tempo médio foi utilizado para calcular Speedup ($T1/Tp$) e Eficiência ($Speedup/p$).

Característica	Ambiente experimental
Sistema operacional	Ubuntu 24.04.5 LTS (Noble Numbat)
Kernel	Linux 6.8.0-1064-azure
Arquitetura	x86_64
Processador identificado	AMD EPYC 7763 64-Core Processor
CPUs lógicas disponíveis	2
Núcleos expostos	1
Threads por núcleo	2
Frequência observada	aprox. 3,24 GHz
Cache	L1d 32 KiB; L1i 32 KiB; L2 512 KiB; L3 32 MiB
Memória RAM	7,8 GiB (sem swap)
GCC / glibc	13.3.0 / 2.39
Python	3.14.2 - GIL habilitado
Flags de compilação	-O2 -Wall -pthread

O Codespaces disponibilizou apenas duas CPUs lógicas. Essa limitação é importante na interpretação dos resultados, pois criar 4 ou 8 threads de software não significa possuir 4 ou 8 CPUs executando em paralelo.

Evidência de compilação:

```
@PedroSZequi →/workspaces/CompParalela (main) $ make
gcc -O2 -Wall -pthread parte1/contacorrente_seq.c -o parte1/contacorrente_seq
gcc -O2 -Wall -pthread parte1/contacorrente_f1.c -o parte1/contacorrente_f1
gcc -O2 -Wall -pthread parte1/contacorrente_f2.c -o parte1/contacorrente_f2
gcc -O2 -Wall -pthread parte2/primos.c -o parte2/primos
gcc -O2 -Wall -pthread parte2/monte_carlo_pi.c -o parte2/monte_carlo_pi
gcc -O2 -Wall -pthread parte2/soma_vetores.c -o parte2/soma_vetores
@PedroSZequi →/workspaces/CompParalela (main) $ make clean
rm -f parte1/contacorrente_seq
rm -f parte1/contacorrente_f1
rm -f parte1/contacorrente_f2
rm -f parte2/primos
rm -f parte2/monte_carlo_pi
rm -f parte2/soma_vetores
```

3. Evidências Experimentais

3.1 Parte 1 - Conta Corrente

A Parte 1 compara uma versão sequencial, uma versão concorrente sem sincronização e uma versão protegida por mutex. O saldo matematicamente esperado é R\$ 150.001.000,00.

Execução	Saldo final	Tempo
1	R\$ 150.001.000,00	0,0942 s
2	R\$ 150.001.000,00	0,0953 s
3	R\$ 150.001.000,00	0,0944 s
4	R\$ 150.001.000,00	0,0943 s
5	R\$ 150.001.000,00	0,0972 s
Média	R\$ 150.001.000,00	0,0951 s

```
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0942 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0953 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0944 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0943 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_seq
[Sequencial] Saldo final: 150001000.00 | Tempo: 0.0972 s
```

Figura 1 - Cinco execuções da versão sequencial.

Na versão sequencial, as cinco execuções produziram o saldo correto. O tempo médio foi de 0,0951 s, sem ocorrência de disputa sobre a variável saldo.

Execução	Saldo final	Tempo
1	-R\$ 99.999.000,00	0,1053 s
2	-R\$ 99.999.000,00	0,0564 s
3	R\$ 250.001.000,00	0,0490 s
4	R\$ 250.001.000,00	0,0507 s
5	R\$ 250.001.000,00	0,0523 s
Média	resultado inconsistente	0,0627 s

```
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.1053 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: -99999000.00 | Tempo: 0.0564 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0490 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0507 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f1
[Fase 1 - Race Condition] Saldo final: 250001000.00 | Tempo: 0.0523 s
```

Figura 2 - Execuções da Fase 1 sem sincronização.

Na Fase 1, as duas threads alteram o saldo sem sincronização. Os resultados variaram e nenhum deles correspondeu ao valor correto, evidenciando condição de corrida e perda de atualizações.

Execução	Saldo final	Tempo
1	R\$ 150.001.000,00	3,1209 s
2	R\$ 150.001.000,00	3,3358 s
3	R\$ 150.001.000,00	3,5828 s
4	R\$ 150.001.000,00	2,8195 s
5	R\$ 150.001.000,00	3,1792 s
Média	R\$ 150.001.000,00	3,2076 s

```
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.1209 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.3358 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.5828 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 2.8195 s
@PedroSZequi →/workspaces/CompParalela/parte1 (main) $ ./contacorrente_f2
[Fase 2 - Mutex] Saldo final: 150001000.00 | Tempo: 3.1792 s
```

Figura 3 - Execuções da Fase 2 com mutex.

Com pthread_mutex_t, o saldo voltou a ser correto e determinístico. O custo foi um aumento expressivo no tempo, pois o lock e o unlock são executados dentro de um laço com milhões de atualizações.

Implementação	Resultado	Tempo médio
Sequencial	R\$ 150.001.000,00	0,0951 s
Concorrente sem mutex	Inconsistente	0,0627 s
Concorrente com mutex	R\$ 150.001.000,00	3,2076 s

3.2 Parte 2 - Problemas com Pthreads

3.2.1 Contagem de Números Primos

O programa contou números primos entre 1 e 5.000.000. A estratégia final distribuiu candidatos ímpares de forma cíclica entre as threads e manteve uma contagem local por thread.

```

===== 1 THREAD(S) =====
[Primos] K: 5000000 | Threads: 1 | Quantidade: 348513 | Tempo: 0.724502 s
[Primos] K: 5000000 | Threads: 1 | Quantidade: 348513 | Tempo: 0.726965 s
[Primos] K: 5000000 | Threads: 1 | Quantidade: 348513 | Tempo: 0.743080 s
[Primos] K: 5000000 | Threads: 1 | Quantidade: 348513 | Tempo: 0.738313 s
[Primos] K: 5000000 | Threads: 1 | Quantidade: 348513 | Tempo: 0.722797 s
===== 2 THREAD(S) =====
[Primos] K: 5000000 | Threads: 2 | Quantidade: 348513 | Tempo: 0.722391 s
[Primos] K: 5000000 | Threads: 2 | Quantidade: 348513 | Tempo: 0.733311 s
[Primos] K: 5000000 | Threads: 2 | Quantidade: 348513 | Tempo: 0.725169 s
[Primos] K: 5000000 | Threads: 2 | Quantidade: 348513 | Tempo: 0.728358 s
[Primos] K: 5000000 | Threads: 2 | Quantidade: 348513 | Tempo: 0.730413 s
===== 4 THREAD(S) =====
[Primos] K: 5000000 | Threads: 4 | Quantidade: 348513 | Tempo: 0.723214 s
[Primos] K: 5000000 | Threads: 4 | Quantidade: 348513 | Tempo: 0.785596 s
[Primos] K: 5000000 | Threads: 4 | Quantidade: 348513 | Tempo: 0.800957 s
[Primos] K: 5000000 | Threads: 4 | Quantidade: 348513 | Tempo: 0.732183 s
[Primos] K: 5000000 | Threads: 4 | Quantidade: 348513 | Tempo: 0.724769 s
===== 8 THREAD(S) =====
[Primos] K: 5000000 | Threads: 8 | Quantidade: 348513 | Tempo: 0.719546 s
[Primos] K: 5000000 | Threads: 8 | Quantidade: 348513 | Tempo: 0.729742 s
[Primos] K: 5000000 | Threads: 8 | Quantidade: 348513 | Tempo: 0.720341 s
[Primos] K: 5000000 | Threads: 8 | Quantidade: 348513 | Tempo: 0.728178 s
[Primos] K: 5000000 | Threads: 8 | Quantidade: 348513 | Tempo: 0.727813 s

```

Figura 4 - Benchmark de primos com 1, 2, 4 e 8 threads.

Threads	Tempo médio	Speedup	Eficiência
1	0,7311 s	1,0000	100,00%
2	0,7279 s	1,0044	50,22%
4	0,7533 s	0,9705	24,26%
8	0,7251 s	1,0083	12,60%

Todas as execuções produziram 348.513 primos. Não houve Speedup significativo. Além do overhead das threads, o ambiente possuía somente duas CPUs lógicas, o que limita o ganho com 4 e 8 threads.

3.2.2 Cálculo de Pi por Monte Carlo

O programa utilizou 100.000.000 de pontos. Cada thread usou uma semente própria com `rand_r()`. A semente também incorporou nanossegundos para evitar repetições em execuções iniciadas no mesmo segundo.

```

===== 1 THREAD(S) =====
[Monte Carlo] N: 100000000 | Threads: 1 | Pi: 3.14163972 | Tempo: 1.240552 s
[Monte Carlo] N: 100000000 | Threads: 1 | Pi: 3.14147632 | Tempo: 1.275160 s
[Monte Carlo] N: 100000000 | Threads: 1 | Pi: 3.14159900 | Tempo: 1.239835 s
[Monte Carlo] N: 100000000 | Threads: 1 | Pi: 3.14155044 | Tempo: 1.247860 s
[Monte Carlo] N: 100000000 | Threads: 1 | Pi: 3.14163820 | Tempo: 1.247732 s
===== 2 THREAD(S) =====
[Monte Carlo] N: 100000000 | Threads: 2 | Pi: 3.14156944 | Tempo: 0.853525 s
[Monte Carlo] N: 100000000 | Threads: 2 | Pi: 3.14166604 | Tempo: 0.767610 s
[Monte Carlo] N: 100000000 | Threads: 2 | Pi: 3.14169764 | Tempo: 0.748404 s
[Monte Carlo] N: 100000000 | Threads: 2 | Pi: 3.14163968 | Tempo: 0.876095 s
[Monte Carlo] N: 100000000 | Threads: 2 | Pi: 3.14148796 | Tempo: 0.781453 s
===== 4 THREAD(S) =====
[Monte Carlo] N: 100000000 | Threads: 4 | Pi: 3.14160236 | Tempo: 0.855530 s
[Monte Carlo] N: 100000000 | Threads: 4 | Pi: 3.14171000 | Tempo: 0.873550 s
[Monte Carlo] N: 100000000 | Threads: 4 | Pi: 3.14179624 | Tempo: 0.740756 s
[Monte Carlo] N: 100000000 | Threads: 4 | Pi: 3.14160332 | Tempo: 0.740426 s
[Monte Carlo] N: 100000000 | Threads: 4 | Pi: 3.14186104 | Tempo: 0.776229 s
===== 8 THREAD(S) =====
[Monte Carlo] N: 100000000 | Threads: 8 | Pi: 3.14142680 | Tempo: 0.739761 s
[Monte Carlo] N: 100000000 | Threads: 8 | Pi: 3.14171688 | Tempo: 0.757595 s
[Monte Carlo] N: 100000000 | Threads: 8 | Pi: 3.14192012 | Tempo: 0.868125 s
[Monte Carlo] N: 100000000 | Threads: 8 | Pi: 3.14152864 | Tempo: 0.825056 s
[Monte Carlo] N: 100000000 | Threads: 8 | Pi: 3.14187624 | Tempo: 0.737385 s

```

Figura 5 - Benchmark final do método de Monte Carlo.

Threads	Tempo médio	Speedup	Eficiência
1	1,2502 s	1,0000	100,00%
2	0,8054 s	1,5523	77,61%
4	0,7973 s	1,5681	39,20%
8	0,7856 s	1,5915	19,89%

O Monte Carlo apresentou ganho claro ao passar de uma para duas threads. Depois disso, o ganho adicional foi pequeno, o que é coerente com a disponibilidade de apenas duas CPUs lógicas e com os custos de escalonamento.

3.2.3 Soma de Vetores

Foram somados vetores com 50.000.000 de elementos. O trabalho foi dividido em blocos contíguos, e cada thread escreveu em posições exclusivas do vetor C, sem necessidade de mutex.

```

===== 1 THREAD(S) =====
[Soma Vetores] N: 50000000 | Threads: 1 | Resultado: correto | Tempo: 0.164813 s
[Soma Vetores] N: 50000000 | Threads: 1 | Resultado: correto | Tempo: 0.191035 s
[Soma Vetores] N: 50000000 | Threads: 1 | Resultado: correto | Tempo: 0.179488 s
[Soma Vetores] N: 50000000 | Threads: 1 | Resultado: correto | Tempo: 0.150960 s
[Soma Vetores] N: 50000000 | Threads: 1 | Resultado: correto | Tempo: 0.141414 s
===== 2 THREAD(S) =====
[Soma Vetores] N: 50000000 | Threads: 2 | Resultado: correto | Tempo: 0.145782 s
[Soma Vetores] N: 50000000 | Threads: 2 | Resultado: correto | Tempo: 0.107022 s
[Soma Vetores] N: 50000000 | Threads: 2 | Resultado: correto | Tempo: 0.106616 s
[Soma Vetores] N: 50000000 | Threads: 2 | Resultado: correto | Tempo: 0.143599 s
[Soma Vetores] N: 50000000 | Threads: 2 | Resultado: correto | Tempo: 0.102725 s
===== 4 THREAD(S) =====
[Soma Vetores] N: 50000000 | Threads: 4 | Resultado: correto | Tempo: 0.113238 s
[Soma Vetores] N: 50000000 | Threads: 4 | Resultado: correto | Tempo: 0.106638 s
[Soma Vetores] N: 50000000 | Threads: 4 | Resultado: correto | Tempo: 0.123430 s
[Soma Vetores] N: 50000000 | Threads: 4 | Resultado: correto | Tempo: 0.140954 s
[Soma Vetores] N: 50000000 | Threads: 4 | Resultado: correto | Tempo: 0.137380 s
===== 8 THREAD(S) =====
[Soma Vetores] N: 50000000 | Threads: 8 | Resultado: correto | Tempo: 0.105197 s
[Soma Vetores] N: 50000000 | Threads: 8 | Resultado: correto | Tempo: 0.102177 s
[Soma Vetores] N: 50000000 | Threads: 8 | Resultado: correto | Tempo: 0.100187 s
[Soma Vetores] N: 50000000 | Threads: 8 | Resultado: correto | Tempo: 0.103906 s
[Soma Vetores] N: 50000000 | Threads: 8 | Resultado: correto | Tempo: 0.108244 s

```

Figura 6 - Benchmark da soma de vetores.

Threads	Tempo médio	Speedup	Eficiência
1	0,1655 s	1,0000	100,00%
2	0,1211 s	1,3664	68,32%
4	0,1243 s	1,3315	33,29%
8	0,1039 s	1,5926	19,91%

O Speedup ficou abaixo do linear. A soma de vetores é limitada por acesso à memória: várias threads passam a disputar a largura de banda disponível para ler A e B e escrever C.

3.3 Parte 3 - Python e GIL

```
===== PYTHON SEQUENCIAL =====
[Python Sequencial] Tempo: 4.9912 segundos
[Python Sequencial] Tempo: 4.7840 segundos
[Python Sequencial] Tempo: 4.8745 segundos
[Python Sequencial] Tempo: 4.8191 segundos
[Python Sequencial] Tempo: 4.9145 segundos
===== PYTHON THREADS =====
[Python com Threads] Tempo: 4.8785 segundos
[Python com Threads] Tempo: 5.0026 segundos
[Python com Threads] Tempo: 4.8688 segundos
[Python com Threads] Tempo: 5.0097 segundos
[Python com Threads] Tempo: 4.8186 segundos
```

Figura 7 - Cinco execuções do Python sequencial e com duas threads.

Implementação	Tempo médio
Python sequencial	4,8767 s
Python com duas threads	4,9156 s

O GIL foi verificado como habilitado no Python 3.14.2. A versão com duas threads não acelerou a tarefa CPU-bound e ficou aproximadamente 0,8% mais lenta em média. Isso ocorre porque as threads não executam bytecode Python simultaneamente quando o GIL está ativo.

4. Comparação de Desempenho

Problema	1 thread	2 threads	4 threads	8 threads	Speedup máx.
Primos	0,7311 s	0,7279 s	0,7533 s	0,7251 s	~1,01x
Monte Carlo Pi	1,2502 s	0,8054 s	0,7973 s	0,7856 s	~1,59x
Soma de Vetores	0,1655 s	0,1211 s	0,1243 s	0,1039 s	~1,59x

Os resultados mostram que o aumento do número de threads não garante aumento proporcional de desempenho. O ambiente possuía apenas duas CPUs lógicas, e cada problema também apresenta gargalos próprios: balanceamento de trabalho nos primos, overhead de threads no Monte Carlo e largura de banda de memória na soma de vetores.

5. Respostas às Questões de Reflexão

1. Ciclo de Leitura-Modificação-Escrita

A operação `saldo += valor` não é uma única operação indivisível. Em termos simplificados, o processador carrega o valor de saldo para um registrador, realiza a soma e grava o resultado de volta na memória/cache. Duas threads podem ler o mesmo valor antigo ao mesmo tempo e depois uma gravação sobrescrever a outra, causando lost updates. A coerência de cache não transforma essa sequência em uma operação atômica.

2. Impacto e Granularidade do Mutex

O `pthread_mutex_t` permite que apenas uma thread entre na região crítica por vez. Isso elimina atualizações concorrentes e recupera o saldo correto. Porém, colocar lock e unlock dentro de milhões de iterações gera contenção, espera e overhead de sincronização. No experimento, a versão com mutex teve média de 3,2076 s, muito acima da versão sequencial de 0,0951 s.

3. Estratégias Alternativas de Sincronização

Uma alternativa é cada thread calcular suas alterações em uma variável local e realizar apenas uma redução final protegida por mutex. Outra opção é usar tipos e operações atômicas do C11, como `_Atomic` e `atomic_fetch_add`. As duas estratégias evitam realizar um lock tradicional em cada atualização.

4. Decomposição e Balanceamento de Carga

Nos primos foi usada distribuição cíclica dos candidatos ímpares; no Monte Carlo os pontos foram divididos de forma aproximadamente uniforme; e na soma de vetores foram usados blocos contíguos. Se uma thread recebe muito mais trabalho que as outras, as demais terminam antes e ficam esperando, reduzindo a eficiência do paralelismo.

5. Reentrância e Thread-Safety

`rand()` utiliza estado interno compartilhado e pode introduzir sincronização/contenda quando várias threads geram números ao mesmo tempo. `rand_r()` recebe a semente por referência, permitindo que cada thread mantenha seu próprio estado. Uma função reentrante pode ser usada simultaneamente sem depender de um estado global compartilhado da mesma forma.

6. Escalabilidade e Limitações Físicas de Memória

Os resultados não apresentaram Speedup linear. Na soma de vetores, cada operação aritmética é simples, mas exige leitura de dois vetores e escrita de um terceiro. Com várias threads, a largura de banda da memória passa a ser um limite. Além disso, o Codespaces forneceu apenas duas CPUs lógicas, o que restringiu a execução paralela de 4 e 8 threads.

7. O Papel do GIL no CPython

O GIL é um mecanismo do CPython que, quando habilitado, permite que apenas uma thread execute bytecode Python por vez. Historicamente ele simplificou partes do gerenciamento de memória, incluindo a contagem de referências. Por isso, tarefas CPU-bound usando threading não obtêm paralelismo real de bytecode. No teste, o sequencial teve média de 4,8767 s e as duas threads 4,9156 s.

8. Paralelismo Real em Python

O módulo `multiprocessing` cria processos separados, cada um com seu próprio interpretador e espaço de memória, contornando o GIL para tarefas CPU-bound. Em contrapartida, criar processos custa mais, a memória não é compartilhada diretamente e a troca de dados pode exigir serialização com `pickle` e comunicação entre processos (IPC).

9. Lei de Amdahl

A Lei de Amdahl pode ser escrita como $S(p) = 1 / (s + (1-s)/p)$, onde s é a fração efetivamente não paralelizável. Usando como exemplo o Monte Carlo com 8 threads e Speedup observado de aproximadamente 1,59, obtém-se uma fração serial efetiva próxima de 57,5%. Nesse caso, o limite estimado com infinitos processadores seria cerca de $1/0,575 = 1,74$ vezes. Esse valor inclui não apenas código serial, mas também os overheads e limitações do ambiente, portanto deve ser interpretado como uma estimativa prática.

6. Dificuldades Técnicas e Soluções

- No primeiro particionamento dos primos, números pares e ímpares eram distribuídos juntos. Como os pares eram descartados rapidamente, houve desbalanceamento. A solução foi distribuir apenas candidatos ímpares e tratar o número 2 separadamente.
- No Monte Carlo, o uso inicial de `time(NULL)` na semente fez algumas execuções repetirem a mesma sequência. A solução foi incorporar nanossegundos obtidos com `clock_gettime`.
- O GitHub Codespaces disponibilizou apenas duas CPUs lógicas. Isso limitou os ganhos ao testar 4 e 8 threads e foi considerado na análise dos resultados.
- A versão da conta corrente com mutex ficou muito mais lenta porque o lock foi colocado dentro do laço de milhões de iterações, mostrando o custo de uma região crítica de granularidade muito fina.

7. Declaração e Análise do Uso de IA

Foi utilizada uma ferramenta de IA generativa (ChatGPT) como apoio durante o laboratório. O uso incluiu organização do ambiente no GitHub Codespaces e do repositório, sugestões para README e Makefile, orientação de comandos de compilação e execução, apoio na organização das medições e cálculo das tabelas de tempo, Speedup e Eficiência, organização da estrutura do relatório, além de esclarecimentos e revisão de conceitos teóricos.

A IA também foi utilizada para revisar trechos de implementação, quando retornados erros que não sabia resolver ou, mesmo com sucesso, entender se era possível fazer de outra forma mais simples ou mais eficiente, e auxiliar na depuração. As sugestões foram executadas e validadas no terminal antes de serem mantidas no projeto. Entre os pedidos realizados estavam: organização das pastas do repositório, compilação com Pthreads, divisão do trabalho entre threads, uso de `rand_r()`, cálculo de Speedup e explicação do GIL.

O uso da IA exigiu validação crítica. Duas sugestões iniciais precisaram de ajustes durante os testes: a distribuição dos primos causou desbalanceamento entre pares e ímpares, e a primeira semente do Monte Carlo podia repetir valores por utilizar apenas `time(NULL)`. Esses problemas foram identificados a partir das execuções e corrigidos. Assim, a principal aprendizagem foi que as respostas da IA precisam ser verificadas por compilação, execução e comparação dos resultados, e não aceitas automaticamente.

8. Conclusão e Referências

O laboratório permitiu observar que concorrência e paralelismo exigem atenção tanto à correção quanto ao desempenho. A condição de corrida mostrou que uma versão aparentemente rápida pode produzir resultados incorretos. O mutex recuperou a consistência, mas demonstrou o custo de sincronização excessiva. Nos problemas da Parte 2, o ganho de desempenho variou de acordo com o tipo de carga e com os limites do ambiente. Por fim, o teste em Python mostrou que threads não aceleraram uma tarefa CPU-bound com o GIL habilitado.

Referências

- Linux man-pages. pthreads(7) e pthread_mutex_lock(3).
- Python Software Foundation. Documentação do Python 3 - threading e multiprocessing.
- Material didático da disciplina de Computação Paralela - Laboratório Pthreads.